CSC 345 - Project #3: Report

Requirement 1: Correctly read the input logical addresses

```
38      /* reading in files */
39      FILE *address = fopen(argv[1], "r"); /* opening addresses.txt */
```

```
49      /* logical addresses from files */
50      while (fscanf(address, "%d", &logAddr) == 1) {
```

Figure 1-2. Reading in the input logical addresses

The program opens the file from the terminal using fopen() and reads each address using fscanf() inside a while loop.

Requirement 2: Correctly translated the input addresses into physical addresses

```
53          int masked = logAddr & 0xFFFF; /* rightmost 16 bits masked */
54          int pageNum = (masked >> 8) & 0xFF; /* get page number */
55          int offset = masked & 0xFF; /* get offset */
```

```
85              int physicalAddress = frameNum * PAGE_SIZE + offset;
```

```
90              fprintf(out2, "%d\n", physicalAddress);
```

Figure 3-5. Translating the input addresses to physical addresses

Each logical address is masked to 16 bits, split into an 8 bit page number and offset, and then the physical address is computed.

Requirement 3: Correctly retrieved the values stored in the physical addresses

```
74              fseek(backing, pageNum * PAGE_SIZE, SEEK_SET);
75              fread(physicalMem[nextFrame], sizeof(signed char), PAGE_SIZE, backing);
```

```
90          signed char value = physicalMem[frameNum][offset];
```

```
95              fprintf(out3, "%d\n", value);
```

Figure 6-8. Retrieve values stored in the physical addresses

When there is a page fault, 256 bytes are read from the BACKING_STORE.bin into the physical memory, where the signed byte is retrieved.

Requirement 4: Implemented FIFO-based TLB update (main1.c)

```
33      for (int i = 0; i < TLB_SIZE; i++) {
34          TLBPage[i] = -1;
35          TLBFrame[i] = -1;
36      }
```

```
59          /* search TLB for page num */
60          for (i = 0; i < TLB_SIZE; i++) {
61              if (TLBPage[i] == pageNum) {
62                  frameNum = TLBFrame[i];
63                  TLBHits++;
64                  break;
65              }
66          }
```

```
83              TLBPage[TLBNext] = pageNum;
84              TLBFrame[TLBNext] = frameNum;
85              TLBNext = (TLBNext + 1) % TLB_SIZE;
```

Figure 9-11. implement FIFO based TLB update (main1.c)

The 16 entries from the TLB are initialized to -1 and searched first on every memory access. When a miss occurs, the TLB is updated using FIFO replacement.

Requirement 5: Correctly counted the number of page faults

```
30      for (int i = 0; i < PAGE_COUNT; i++) {
31          pageTable[i] = -1;
32      }
```

```
if (pageTable[pageNum] == -1) {
    pageFaults++; /* page fault to load from backing */
```

Figure 12-13. Counted the number of page faults

Every time a page number is found to be -1 in the page table, a page fault is counted and the page is loaded from BACKING_STORE.bin into the physical memory.

Requirement 6: Correctly counted the number of TLB hits

```
64              if (TLBPage[i] == pageNum) {
65                  frameNum = TLBFrame[i];
66                  TLBHits++; /* Requirement 6: count the number of TLB hits */
67                  break;
68              }
```

Figure 14. Counted the number of TLB hits

Every time the TLB is searched, and a matching page number is found, TLBHits is incremented and the total hit count and rate are printed.

Requirement 7: Implemented FIFO-based page replacement algorithm (main2.c)

```
77              /* requirement 7: implement FIFO based page replacement */
78              if (nextFrame < FRAME_COUNT) { /* free frames available */
79                  frame = nextFrame++;
80              }
81              else { /* no free frames */
82                  int evictedPage = FIFO_queue[FIFO_head];
83                  frame = pageTable[evictedPage];
84
85                  pageTable[evictedPage] = -1;
86
87                  for (i = 0; i < TLB_SIZE; i++) {
88                      if (TLBPage[i] == evictedPage) {
89                          TLBPage[i] = -1;
90                          TLBFrame[i] = -1;
91                          break;
92                      }
93                  }
94                  /* FIFO head to next oldest page */
95                  FIFO_head = (FIFO_head + 1) % FRAME_COUNT;
```

Figure 15. Implemented FIFO based page replacement algorithm

In main2.c, a frame number of 128 is used. When all frames are full, a circular FIFO queue gets rid of the oldest page.

Requirement 8: Implemented LRU-based page replacement algorithm (main3.c)

```
80              int LRU_frame = 0;
81              for (int i = 1; i < FRAME_COUNT; i++) {
82                  if (LRU_counter[i] < LRU_counter[LRU_frame]) {
83                      LRU_frame = i;
84                  }
85              }
86
87              frame = LRU_frame;
88
89              int evictedPage = frame_to_page[frame];
90              pageTable[evictedPage] = -1;
91
92              for (i = 0; i < TLB_SIZE; i++) {
93                  if (TLBPage[i] == evictedPage) {
94                      TLBPage[i] = -1;
95                      TLBFrame[i] = -1;
96                      break;
97                  }
98              }
```

Figure 16. Implemented LRU based page replacement algorithm

Similarly to main2.c, main3.c uses 128 frames and gets rid of the least recently used page. This is done by tracking the last access time for each frame.

Requirement 9: Test different number of frames for FIFO and LRU page replacement

Frame #	FIFO Page Fault/Rate		LRU Page Fault/Rate	
32	882/1000	0.88	882/1000	0.88
64	762/1000	0.76	762/1000	0.76
128	538/1000	0.54	537/1000	0.54
256	244/1000	0.24	244/1000	0.24
512	244/1000	0.24	244/1000	0.24
768	244/1000	0.24	244/1000	0.24
1024	244/1000	0.24	244/1000	0.24

Figure 17. Table containing FIFO and LRU page fault and rate

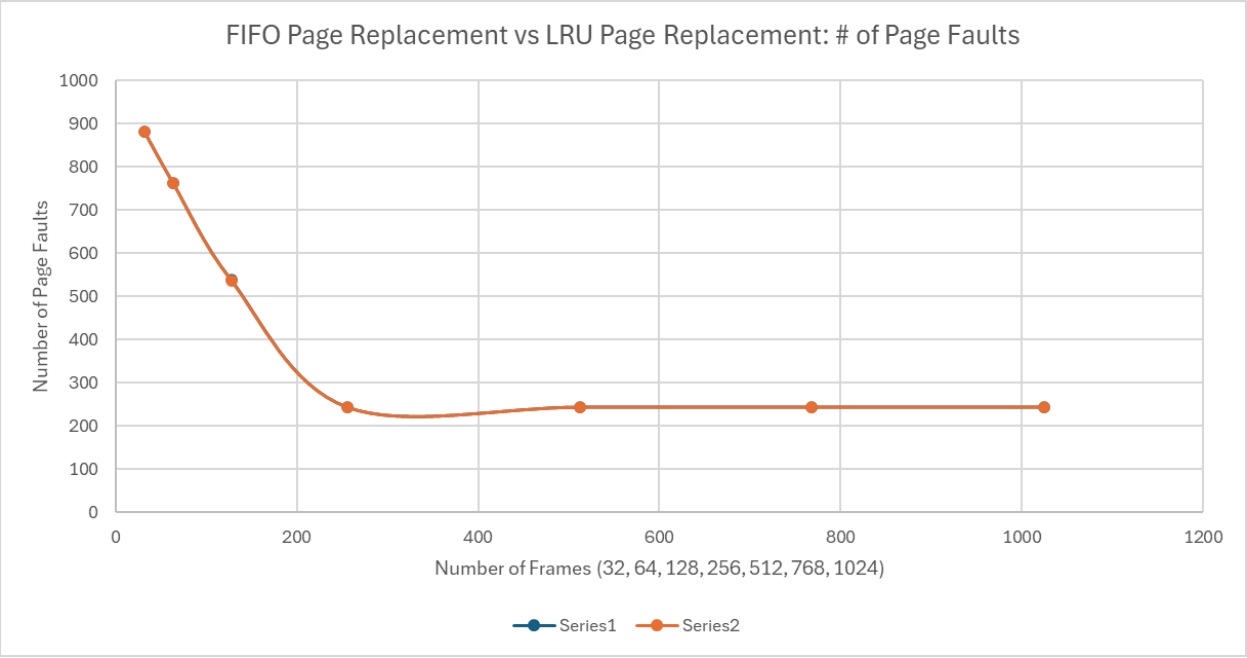

Figure 18. Plot for FIFO and LRU page fault

As seen in figures 17 and 18, the results for the number of page faults are practically identical for FIFO and LRU page replacement algorithms. In the plot, Series1 (blue) is hardly seen, as the Series1 and Series2 almost entirely overlap on the plot. Furthermore, the only difference in the table is when the frame number is 128. In this case, the FIFO number of page faults is 538, and the LRU number of page faults is 537. They are the same because addresses.txt is randomly generated and exhibits no memory access locality. For both algorithms, the higher the frame, the lower the page faults. This trend continues until the number of frames hits 256, and the page faults levels out at 244 for the rest of the tests.